

Кросплатформне програмування

10

Оброблення виняткових
ситуацій



Оброблення виняткових ситуацій

Про що йтиметься:

- Перехоплення та оброблення помилок
- Принципи оброблення винятків
- Вкладені try-catch блоки
- Використання об'єкта винятку
- Генерування винятків
- Контрольовані та неконтрольовані винятки
- Створення власних класів винятків



Перехоплення та оброблення помилок

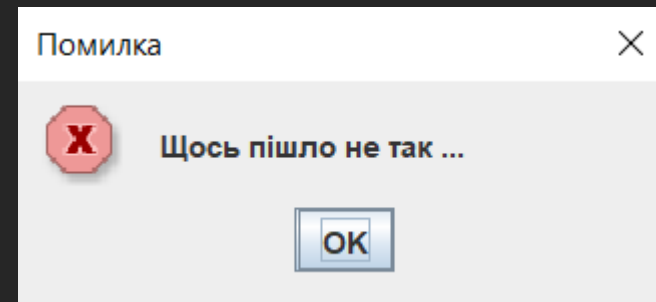
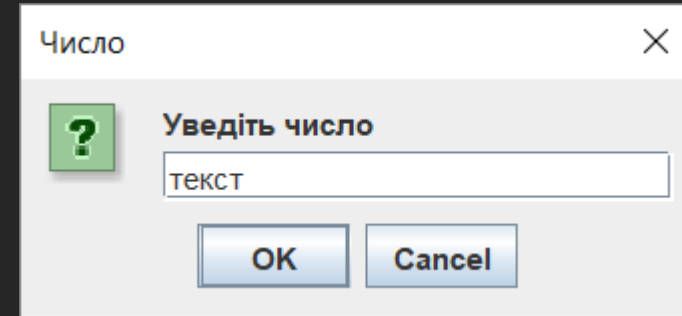
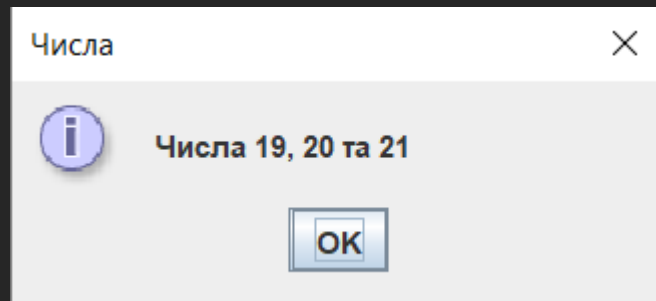
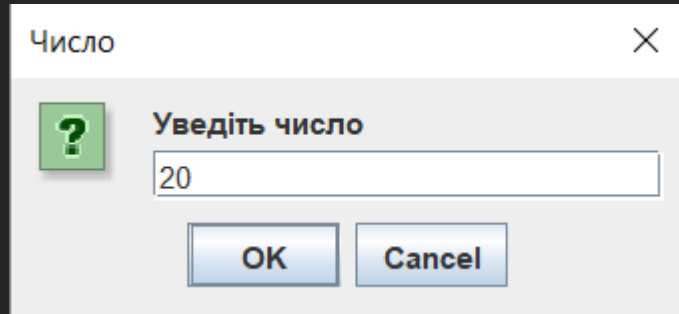
Оснoву системи обробки виняткових ситуацій становить ієрархія класів, що описують виняткові ситуації, та оператор try-catch

```
// Статичний імпорт:
import static javax.swing.JOptionPane.*;
import static java.lang.Integer.*;
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Текстова змінна для запису
        // значення, що зчитується:
        String input;
        // Змінна для запису цілого числа:
        int num;
        // Відображення діалогового кона із полем введення:
        input=showInputDialog (null,
            "Уведіть число", // Текст над полем введення
            "Число", // Назва вікна
            QUESTION_MESSAGE // Тип вікна
        );
    }
}
```

```
// Блок контрольованого коду:
try{
    // Спроба перетворити текст на число:
    num=parseInt(input);
    // Відображення діалогового вікна з числами:
    showMessageDialog (null,
        // Відображається у вікні текст:
        "Числа "+(num-1)+"", "+num+" та "+(num+1)",
        "Числа", // Назва вікна
        INFORMATION_MESSAGE // Тип вікна
    );
    // Обробка помилок (код виконується,
    // якщо у контрольному блоці виникла помилка):
}catch(Exception e){
    // Відображення діалогового вікна:
    showMessageDialog (null,
        "Щось пішло не так ...", // Повідомлення у вікні
        "Помилка", // Назва вікна
        ERROR_MESSAGE // Тип вікна
    );
}
}
```



Перехоплення та оброблення помилок (результат)





Ієрархія класів помилок

- Java має ієрархію класів, які відповідають помилкам певного типу. У вершині ієрархії знаходиться клас **Throwable**
- Throwable має два підкласи: **Error** та **Exception**. Клас **Error** відповідає "фатальним" помилкам, які зазвичай програмними методами не обробляються
- У класу **Exception** багато підкласів, і серед них є клас **RuntimeException**
- Клас **RuntimeException** є суперкласом для багатьох класів помилок



Принципи оброблення винятків

- Якщо помилка виникла, то відповідно до її типу створюється об'єкт (виняток), який передається для обробки методу, який викликав помилку (кажуть, що виняток вкидається в метод)
- Якщо в методі помилка обробляється - добре . Якщо помилка в методі не обробляється, то об'єкт помилки передається для обробки в метод, в якому викликався даний метод, і так по ланцюжку, аж до головного методу програми
- Якщо й у головному методі помилка не обробляється, то у справу вступає так званий обробник за замовчанням - програма достроково припиняє виконання

Реалізація оброблення винятків

- Фрагмент коду, який може спричинити помилку, розміщується в **try-блоці**
- Після **try-блоку** вказують **catch-блоки** (їх може бути декілька)
- Кожен **catch-блок** призначений для обробки помилок певного типу. Під типом помилки мається на увазі клас, на основі якого, у разі виникнення помилки, створюється об'єкт помилки
- У кожному **catch-блоці** в круглих дужках крім класу помилки вказується формальна назва для об'єкта помилки. При необхідності, цей об'єкт може бути використаний у явному вигляді в програмному коді **catch-блоку**
- Якщо під час виконання програмного коду в **try-блоці** помилка не виникає, то всі **catch-блоки** ігноруються
- Якщо в **try-блоці** виникла помилка, то для цієї помилки створюється об'єкт, і далі з метою обробки винятку перевіряються **catch-блоки**. Перевірка виконується щодо збігу класу об'єкта помилки й класу, вказаного в **catch-блоці**. Якщо збіг знайдено, об'єкт помилки передається у відповідний **catch-блок** для обробки
- Важлива обставина полягає в тому, що "збіг" розуміється в широкому сенсі: класи можуть збігатися не буквально - достатньо, щоб клас об'єкта помилки був підкласом класу, вказаного в **catch-блоці**. У цьому сенсі, якщо ми в **catch-блоці** вказуємо ім'я класу **Exception**, то такий **catch-блок** перехоплюватиме фактично всі помилки (які в принципі можна обробити), оскільки класи помилок, з якими ми маємо справу, прямо чи опосередковано є спадкоємцями класу **Exception**
- У **try-catch** конструкції після **catch-блоків** може використовуватися **finally-блок** (блок починається ключовим словом **finally** і виділяється фігурними дужками). Програмний код з **finally-блоку** виконується в будь-якому випадку: і якщо виникла помилка, і якщо вона не виникла. Зазвичай **finally-блоки** корисні при використанні вкладених **try-блоків** (коли всередині **try-блоку** розміщується ще одна **try-catch** конструкція)

Клас винятку	Описання
ArithmeticException	Помилка, пов'язана з виконанням арифметичних операцій
ArrayStoreException	Помилка, пов'язана зі спробою записати в масив об'єкт типу, що не відповідає типу елементів масиву
ClassCastException	Помилка, пов'язана зі спробою некоректного приведення типів
IllegalArgumentException	Помилка, пов'язана з некоректною передачею аргументів методу. Цей клас має багато підкласів. Серед них, зокрема, є клас NumberFormatException , що відповідає помилці, яка виникає при спробі перетворити некоректне текстове представлення для числа в число
IndexOutOfBoundsException	Помилка, пов'язана з некоректно вказаним індексом. Цей клас має два підкласи: клас IndexOutOfBoundsException відповідає помилці, що виникає при неправильно вказаному індексі масиву, а клас StringIndexOutOfBoundsException відповідає помилці, пов'язаній з некоректно вказаним індексом елемента в текстовому рядку
NegativeArraySizeException	Помилка, пов'язана зі спробою створити масив негативного розміру
NoSuchElementException	Помилка, пов'язана зі зверненням до неіснуючого елемента
NullPointerException	Помилка, пов'язана з використанням порожнього посилання (значення null) замість посилання на об'єкт
UnsupportedOperationException	Помилка пов'язана з тим, що операція, що запитується, не підтримується



Програма: постановка задачі

У програмі відображається діалогове вікно, в якому користувачеві пропонується вказати розмір числового масиву

Якщо користувач вводить коректне значення, створюється відповідний числовий масив і заповнюється символами

Послідовність символів з масиву відображається у діалоговому вікні

Після цього користувача просять вказати індекс елемента в масиві, і після зчитування введеного користувачем індексу відображається відповідний символ

Також у програмі відстежуються можливі помилки, пов'язані з тим, що:

- користувач скасував введення числа чи ввів нечислове значення;
- увів від'ємне значення для розміру масиву;
- вказав індекс елемента масиву, який виходить за допустимий діапазон значень індексу

Програма: результат

Символьний масив

Вкажіть розмір масиву

OK Cancel



Символи з масиву

|A|B|C|D|E|F|G|H|I|J|K|L|

OK



Індекс елемента масиву

Вкажіть індекс елемента

OK Cancel



Символ

Індекс - 5
Символ - F

OK

Помилка при виборі індексу

Ви впевнені? Такого елемента немає!

OK

Уведений індекс виходить за допустимий діапазон

ArrayIndexOutOfBoundsException

Помилка

На жаль, виникла помилка...

OK

Уведено нечислове значення для індексу або скасовано введення

NumberFormatException

Помилка при створенні масиву

Некоректний розмір масиву!

OK

При визначенні розмірів масиву вказано від'ємне число

NegativeArraySizeException



Програма: код - 1



```
// Статичний імпорт:
import static javax.swing.JOptionPane.*;
import static java.lang.Integer.*;
// Головний клас:
class Demo{
    public static void main(String[] args){
        // Змінна для зчитування даних із поля введення:
        String input;
        // Змінна масиву:
        char[] symbs;
        // Змінні для запису розміру масиву
        // та індексу елемента:
        int size,index;
        // Відображення вікна з полем уведення для зчитування
        // розміру масиву:
        input=showInputDialog(null,
            "Вкажіть розмір масиву", // Текст над полем введення
            "Символьний масив", // Назва вікна
            QUESTION_MESSAGE // Тип піктограми
        );
        // Контрольований код:
        try{
            // Перетворення тексту на число:
            size=parseInt(input);
```



```
// Створення масиву:
symbs=new char[size];
// Текстова змінна для формування
// текст повідомлення:
String txt="| ";
// Заповнення масиву символами та формування
// Текст для відображення у вікні:
for(int k=0;k<size;k++){
    // Елементу масиву присвоюється значення:
    symbs[k]=(char) ('A'+k);
    // До тексту дописується значення елемента:
    txt+=symbs[k]+" | ";
} // Відображення повідомлення з символами з масиву:
showMessageDialog(null,
    txt, // Повідомлення
    "Символи з масиву", // Назва вікна
    INFORMATION_MESSAGE // Тип повідомлення
); // Відображення вікна з полем уведення для зчитування
// значення індексу елемента масиву:
input = showInputDialog (null,
    // Текст над полем уведення:
    "Вкажіть індекс елемента",
    // Назва вікна:
    "Індекс елемента масиву",
    // Тип піктограми:
    QUESTION_MESSAGE);
```



Програма: код - 2

```
// Перетворення тексту на число:
index=parseInt(input);
// Текст для відображення в діалоговому вікні:
txt="Індекс - "+index+"\nСимвол - "+syms[index];
// Відображення значення елемента масиву:
showMessageDialog(null,
    txt, // Текст повідомлення
    "Символ", // Назва вікна
    INFORMATION_MESSAGE // Тип вікна
);
}
// Обробка виняткових ситуацій.
// Користувач скасував уведення чи ввів
// нечислове значення:
catch(NumberFormatException e){
    // Відображення вікна з повідомленням:
    showMessageDialog(null,
        // Повідомлення:
        "На жаль, виникла помилка...",
        "Помилка", // Назва вікна
        WARNING_MESSAGE // Тип вікна
    );
}
```



```
// Користувач вказав неправильний розмір масиву:
catch(NegativeArraySizeException e){
    // Відображення діалогового вікна:
    showMessageDialog(null,
        // Повідомлення:
        "Некоректний розмір масиву!",
        // Назва вікна:
        "Помилка при створенні масиву",
        // Піктограма:
        ERROR_MESSAGE
    );
} // Користувач вказав неправильне значення
// для індексу елемента масиву:
catch(ArrayIndexOutOfBoundsException e){
    // Відображення діалогового вікна:
    showMessageDialog(null,
        // Повідомлення:
        "Ви впевнені? Такого елемента немає!",
        // Назва вікна:
        "Помилка при виборі індексу",
        // Піктограма:
        QUESTION_MESSAGE
    );
}
}
```





Вкладені блоки try-catch

Блоки try-catch можуть бути вкладеними. У разі якщо помилка виникає у внутрішньому try-блоці, то обробки помилки спочатку проглядаються catch-блоки внутрішнього try-блока (у якому виникла помилка). Якщо помилка внутрішніми catch-блоками не обробляється, то виняток передається для обробки у зовнішніх catch-блоках

Можливі результати:

Ненульове ціле число: **5**
120/5=24
Добре що ви ввели число

Ненульове ціле число: **три**
Слід було ввести число!

Ненульове ціле число: **0**
Помилка: ділення на нуль!
Добре що ви ввели число

```
// Імпорт класів:
import java.util.Scanner;
// Головний клас:
class Demo{
    // Головний метод:
    public static void main(String[] args){
        // Об'єкт класу Scanner (потрібен для консольного введення):
        Scanner input=new Scanner(System.in);
        // Контрольований код (зовнішній блок):
        try{
            System.out.print("Ненульове ціле число: ");
            int num=input.nextInt();
            try{
                // Розрахунки:
                System.out.println("120/"+num+"="+ (120/num) );
            }catch(ArithmeticException e){
                System.out.println("Помилка: ділення на нуль!");
            }finally{
                System.out.println("Добре що ви ввели число");
            }
        }catch(Exception e){
            System.out.println("Слід було ввести число!");
        }
    }
}
```



Об'єкт винятку

В описанні catch-блоків незмінно присутнє, крім імені класу оброблюваного в блоці винятку, ще й формальне позначення об'єкта винятку

Ці об'єкти можна використовувати для обробки помило

Зокрема, об'єкт винятку можна використовувати у виразах як "текстовий" операнда: завдяки перевизначеному методу **toString()** об'єкт винятку в таких випадках автоматично приводиться до текстового формату



Програма: використання об'єкта винятку

У програмі (в головному методі) послідовно розміщено три try-catch конструкції

У try-блоці генерується помилка певного типу, а в catch-блоці ця помилка оброблюється

Обробка помилки полягає в тому, що об'єкт винятку (у всіх трьох catch-блоках позначений як e) передається аргументом методу println()

Наслідки приведення об'єкта винятку до текстового формату можна оцінити за результатом виконання програми

```
class Demo{
    public static void main(String[] args){
        try { // Код із помилкою
            System.out.println("Від'ємний розмір:");
            int[] a= new int[-1];
        } // Обробка помилки:
        catch (NegativeArraySizeException e){
            System.out.println(e);
        } // Код із помилкою:
        try{
            System.out.println("Неправильний індекс:");
            int[] b={1};
            b[-1]=0;
        } // Обробка помилки:
        catch (ArrayIndexOutOfBoundsException e){
            System.out.println(e);
        } // Код із помилкою:
        try{
            System.out.println("Ділення на нуль:");
            int c=10/0;
        } // Обробка помилки:
        catch (ArithmeticException e){
            System.out.println(e);
        }
    }
}
```

Результат:

```
Від'ємний розмір:
java.lang.NegativeArraySizeException: -1
Неправильний індекс:
java.lang.ArrayIndexOutOfBoundsException: Index -1 out of bounds for length 1
Ділення на нуль:
java.lang.ArithmeticException: / by zero
```



Генерування винятків

Для штучного генерування винятків використовується оператор `throw`, після якого вказується об'єкт винятку. Об'єкт винятку створюють спеціально (як створюється будь-який інший об'єкт), або використовую вже згенерований об'єкт, переданий для обробки в `catch`-блок

```
class Demo{
    public static void main(String[] args){
        Exception me=new Exception("Штучна помилка");// Створення об'єкта винятку
        try{                                     // Контрольований код (зовнішній блок)
            System.out.println("Генерується помилка");
            try{                                 // Контрольований код (внутрішній блок)
                throw me;                       // Генерування винятку
            } // Обробка виключення (внутрішній блок):
            catch(Exception one){
                System.out.println(one);
                System.out.println("А тепер ще одна помилка");
                throw one;                      // Повторне генерування винятку
            }
        } // Обробка винятку (зовнішній блок):
        catch(Exception two){
            System.out.println(two);
        }
        System.out.println("Виконання програми завершено");
    }
}
```

Результат:

```
Генерується помилка
java.lang.Exception: Штучна помилка
А тепер ще одна помилка
java.lang.Exception: Штучна помилка
Виконання програми завершено
```




Контрольовані та неконтрольовані винятки

- Усі винятки в Java поділяються на контрольовані та неконтрольовані. До неконтрольованих винятків належать винятки класів, які є підкласами класу **RuntimeException** або класу **Error**. Всі інші винятки відносяться до контрольованих
- Різниця між винятками різних типів полягає в тому, що для контрольованих винятків автоматично виконується перевірка на наявність обробки винятку: якщо метод може згенерувати в процесі виконання контрольований виняток, то для такого винятку в методі повинна бути передбачена обробка
- Якщо метод все ж таки не містить код для обробки контрольованого винятку (але потенційно може такий виняток згенерувати), то в описі методу в явному вигляді вказується, що він може генерувати необроблений контрольований виняток: в описі методу після його імені та списку аргументів (але перед фігурними дужками з кодом методу) вказується ключове слово **throws** і через кому перераховуються класи контрольованих винятків, які метод може згенерувати, але які в тілі методу не обробляються



Контрольовані винятки

Клас винятку	Описання
<code>ClassNotFoundException</code>	Помилка, пов'язана з неможливістю отримання доступу до класу
<code>IllegalAccessException</code>	Помилка під час спроби доступу до ресурсу
<code>InstantiationException</code>	Помилка, пов'язана з неможливістю створення об'єкта (наприклад, на основі абстрактного класу)
<code>InterruptedException</code>	Помилка, пов'язана з перериванням одного потоку іншим потоком
<code>NoSuchFieldException</code>	Помилка, пов'язана з відсутністю поля
<code>NoSuchMethodException</code>	Помилка, пов'язана з відсутністю методу

До уваги

Класи `ClassNotFoundException`, `IllegalAccessException`, `InstantiationException`, `NoSuchFieldException` та `NoSuchMethodException` є підкласами класу `ReflectiveOperationException`, який, у свою чергу, є підкласом класу `Exception`. Клас `InterruptedException` є підкласом класу `Exception`. Усі перелічені класи знаходяться в пакеті `java.lang`

Програма: контрольовані винятки - 1

```
class Demo{
    // Метод викидає контрольований виняток:
    static void alpha(int n) throws Exception{
        // Текст передачі аргументом конструктору
        // при створенні об'єкта винятку:
        String txt="Метод alpha() з аргументом " + n;
        // Генерування винятку:
        throw new Exception(txt);
    }
    // Метод викидає неконтрольований виняток:
    static void bravo(int n){
        // Текст передачі аргументом конструктору
        // при створенні об'єкта виключення:
        String txt="Метод bravo() з аргументом " + n;
        // Контрольований код:
        try{
            if(n>0){
                // Генерування контрольованого винятку:
                throw new Exception(txt);
            }
            else{
                // Генерування неконтрольованого виключення:
                throw new RuntimeException (txt);
            }
        }
    }
}
```

```
    // Обробка неконтрольованого винятку:
    catch(RuntimeException e){
        // Повторне генерування неконтрольованого
        // винятку:
        throw e;
    } // Обробка контрольованого винятку:
    catch(Exception e){
        System.out.println("Обробка помилки в bravo():");
        System.out.println(e.getMessage());
        System.out.println("*****");
    }
} // Метод для виклику під час обробки винятків.
// Аргументом методу передається об'єкт винятку:
static void catchMe(Exception e){
    System.out.println("Обробка помилки в main():");
    System.out.println (e.getMessage());
    System.out.println("-----");
} // Головний метод:
public static void main(String[] args){
    // Контрольований код:
    try{
        // При виклику методу викидається
        // контрольований виняток класу Exception:
        alpha(123);
    }
}
```

Програма: контрольовані винятки - 2

```
// Обробка винятку:
catch(Exception e){
    // Виклик методу для обробки винятку:
    catchMe(e);
} // Контрольований код:
try{
    // При виклику методу не викидається
    // виняток:
    bravo(123);
} // Обробка винятку (блок не використовується,
// оскільки виняток не генерується):
catch (Exception e) {
    // Виклик методу для обробки винятку:
    catchMe(e);
} // Контрольований код:
try{
    // При виклику методу викидається неконтрольований виняток
    // класу RuntimeException:
    bravo(-1);
} // Обробка винятку:
catch(Exception e){
    // Виклик методу обробки виключення:
    catchMe(e);
}
}
```

Результат:

```
Обробка помилки в main():
Метод alpha() з аргументом 123
-----
Обробка помилки в bravo():
Метод bravo() з аргументом 123
*****
Обробка помилки в main():
Метод bravo() з аргументом -1
-----
```

Програма: контрольовані винятки - 3

До уваги

У методі `bravo()` залежно від значення аргументу генерується виняток класу `Exception` або виняток класу `RuntimeException`. Відповідно, в методі передбачені `catch-блоки` для обробки винятків даних класів. Специфіка ситуації в тому, що клас `RuntimeException` є підкласом класу `Exception`. Тому `catch-блок`, описаний для винятку класу `Exception`, міг би обробляти і виняток класу `RuntimeException` якби не було передбачено `catch-блок` спеціально для обробки винятків класу `RuntimeException`

При генеруванні винятку класу `RuntimeException` він повторно генерується в `catch-блоці`, причому з тим самим об'єктом винятку, що й вперше. В результаті метод `bravo()` при недодатніх значеннях аргументу викидає виняток класу `RuntimeException`. Але якщо ми спробуємо досягти того ж ефекту, обмежившись одноразовим генеруванням винятку класу `RuntimeException`, відмовившись від обробки (і повторного генерування) цього винятку в `catch-блоці`, то такий виняток буде оброблено в `catch-блоці`, призначеному для обробки винятків класу `Exception`. І все тому, що клас `RuntimeException` є підкласом класу `Exception`



Користувацькі класи винятків

- Існує можливість створювати користувацькі класи для винятків: на основі одного з класів винятків шляхом спадкування створюється клас
- Об'єкт, створений на основі такого класу, може використовуватися при генеруванні виняткових ситуацій
- Наприклад, як суперклас можемо використовувати клас **Exception**, або, скажімо, клас **RuntimeException**
- У першому випадку отримаємо власний клас винятку контрольованого типу, а в другому - неконтрольованого типу



Програма: користувачькі винятки

```
// Клас контрольованого винятку:
class MyException extends Exception{
    private int code;    // Закрите числове поле
    MyException(int n){  // Конструктор
        super();
        code=n;
    } // Переозначення методу toString():
    public String toString(){
        String txt="Виняток класу MyException\n";
        txt+="Код помилки: "+code+"\n";
        txt+="-----";
        return txt;
    }
}

// Клас неконтрольованого винятку:
class MyMistake extends RuntimeException{
    private int code; // Закрите числове поле
    MyMistake(int n){ // Конструктор
        super();
        code=n;
    } // Переозначення методу toString():
    public String toString(){
        String txt="Виняток класу MyMistake\n";
        txt+="Код помилки: "+code+"\n";
        txt+="*****";
        return txt;
    }
}
```

```
// Головний клас:
class Demo{
    // Статичний метод викидає контрольований
    // виняток класу MyException:
    static void alpha(int n) throws MyException{
        throw new MyException(n);
    } // Статичний метод викидає неконтрольований
    // виняток класу MyMistake:
    static void bravo(int n){
        throw new MyMistake(n);
    } // Головний метод:
    public static void main(String[] args){
        try {                // Контрольований код (зовнішній блок)
            try{              // Контрольований код (внутрішній блок)
                alpha(123);   // Метод викидає виняток MyException
            } // Обробка винятку класу MyException:
            catch(MyException e){
                System.out.println(e);
                bravo(321);
            }
        } // Обробка винятку класу MyMistake:
        catch(MyMistake e) {
            System.out.println(e);
        }
    }
}
```

Результат:

Виняток класу MyException
Код помилки: 123

Виняток класу MyMistake
Код помилки: 321

Дякую за увагу!

Запитання?